

Para conectar dos microservicios en Spring Boot de forma desacoplada, utilizaremos **OpenFeign**. Es la herramienta más estándar y sencilla para que un microservicio le pida datos a otro como si fuera una función local.

Vamos a usar el ejemplo de una **Tienda de Tecnología**:

1. **Microservicio de Stock**: Tiene el inventario.
 2. **Microservicio de Ordenes**: Necesita consultar al de Stock antes de vender.
-

1. Microservicio de STOCK (El que entrega los datos)

Este microservicio es "independiente". No sabe quién lo llama, solo expone un endpoint.

Repositorio (StockRepository.java):

Java

```
public interface StockRepository extends JpaRepository<Stock, Long> {  
    // Busca stock por el ID del producto  
    Optional<Stock> findByProductoId(Long productoId);  
}
```

Servicio (StockService.java):

Java

```
@Service  
public class StockService {  
    @Autowired  
    private StockRepository repo;  
  
    public boolean tieneStock(Long id) {  
        return repo.findByProductoId(id)  
            .map(s -> s.getCantidad() > 0)  
            .orElse(false);  
    }  
}
```

Controlador (StockController.java):

Java

```
@RestController  
@RequestMapping("/api/stock")  
public class StockController {  
    @Autowired  
    private StockService service;  
  
    @GetMapping("/validar/{id}")  
    public ResponseEntity<Boolean> validar(@PathVariable Long id) {  
        return ResponseEntity.ok(service.tieneStock(id));  
    }  
}
```

2. Microservicio de ORDENES (El que pide los datos)

Para conectar, este microservicio necesita la dependencia spring-cloud-starter-openfeign.

El Cliente Feign (StockClient.java): Esta es la "interfaz mágica". Nota que la URL debe coincidir con el nombre con el que el Microservicio de Stock se registró en **Eureka**.

Java

```
// "ms-stock" es el nombre del microservicio en el archivo properties/yml
@FeignClient(name = "ms-stock")
public interface StockClient {

    // Debe ser EXACTAMENTE igual a la firma del controlador del otro microservicio
    @GetMapping("/api/stock/validar/{id}")
    boolean consultarDisponibilidad(@PathVariable("id") Long id);
}
```

Servicio (OrdenService.java):

Aquí es donde ocurre la conexión real. El servicio de órdenes usa al cliente Feign.

Java

```
@Service
public class OrdenService {

    @Autowired
    private OrdenRepository repo;

    @Autowired
    private StockClient stockClient; // Inyectamos el cliente Feign

    public Orden crearOrden(Orden orden) {
        // Llamada remota al otro microservicio via Feign
        boolean hayStock = stockClient.consultarDisponibilidad(orden.getProductoId());

        if (hayStock) {
            return repo.save(orden);
        } else {
            return null; // O lanzar una excepción personalizada
        }
    }
}
```

Controlador (OrdenController.java):

Java

```
@RestController
@RequestMapping("/api/ordenes")
public class OrdenController {
    @Autowired
    private OrdenService service;

    @PostMapping
    public ResponseEntity<?> guardar(@RequestBody Orden o) {
        Orden nueva = service.crearOrden(o);
        if (nueva != null) {
            return ResponseEntity.status(201).body(nueva);
        }
        return ResponseEntity.status(400).body("Error: No hay stock disponible pa
```

Resumen de la conexión para tu proyecto:

1. **Independencia:** Cada uno tiene su propio pom.xml, su propia base de datos y su propio puerto (ej: Stock en 8081 y Ordenes en 8082).
2. **Eureka:** Ambos deben tener la anotación @EnableDiscoveryClient en su clase principal para que el **Gateway** y **Feign** puedan encontrarlos por nombre (ms-stock) y no por IP.
3. **Feign:** El microservicio que "necesita datos" es el que lleva la interfaz @FeignClient.

¿Te queda claro cómo la interfaz StockClient actúa como un "puente" entre ambos servicios sin que el código se vuelva complejo?